

SAE 5.01

Rapport de semaine 09/10/2026

Avancement au niveau du modèle :

Cette semaine, on a réorganisé tout le code Python de l'IA. Comme prévu dans le rapport précédent, les scripts ont été transformés en classes pour que le code soit plus clair :

- DatasetAnalyzer pour analyser le dataset ;
- DatasetPreparer pour préparer le petit dataset ;
- MobileNetTrainer pour entraîner le modèle ;
- CarPredictor pour prédire une image.

Un fichier main.py propose maintenant un menu pour lancer chacune de ces actions. Les anciens scripts (train.py, prepare_dataset.py) ont été supprimés.

Le petit dataset (dataset_small) est maintenant directement dans le dépôt Git : 6 voitures, 920 images (734 pour l'entraînement et 186 pour le test). Il n'est donc plus nécessaire de télécharger le dataset complet de 6 Go pour tester le modèle.

Un script d'évaluation (evaluate.py) a aussi été ajouté. Il évalue le modèle déjà sauvegardé sans le réentraîner et génère tout seul la matrice de confusion, les erreurs classe par classe et des exemples d'images mal reconnues. Sur les 186 images de test, il donne :

- 79,6 % de bonnes réponses en choisissant la voiture la plus probable ;
- 96,8 % si on accepte les 3 meilleures propositions.

Voiture	Rappel (part des photos retrouvées)
Citroën 2CV	100 %
Jeep Grand Cherokee	95 %
Ford Mustang	83 %
Fiat 500 Abarth	83 %
Toyota Hilux	68 %
Ferrari F12	47 %

La Ferrari F12 reste la voiture la plus difficile à évaluer: 10 photos sur 30 sont confondues avec la Ford Mustang.

Avancement au niveau de l'application :

Cette semaine, l'IA a été branchée sur l'application.

Service IA. Un petit serveur Python (Flask, fichier ai/server.py) charge le modèle. Il reçoit une image et renvoie la voiture reconnue avec sa confiance. Le backend Node.js l'appelle quand on clique sur « Analyser », ce qui garde la même architecture : l'application Flutter parle uniquement au backend. Pour cela, on a ajouté :

- une route /api/predict, accessible sans compte ;
- un service recognition.js qui transforme le nom de la classe (par exemple FERRARI_F12_Berlinetta_20122015) en marque, modèle et années de production ;
- un service imageUpload.js qui vérifie l'image reçue. Il sert aux deux routes, ce qui évite d'écrire deux fois la même vérification.

Si le service IA est éteint, l'application affiche « Service de reconnaissance indisponible » et rien n'est enregistré.

Cadre rouge autour de la voiture. Le sujet demande un cadre autour de l'objet détecté. Le modèle dit quelle voiture c'est, mais pas où elle est dans la photo. On a donc ajouté un détecteur d'objets (SSD MobileNet entraîné sur COCO, au format TensorFlow Lite), fichier ai/src/detector.py. Il trouve la voiture et renvoie un rectangle, qui est dessiné en rouge sur la photo (widget FramedImage). Sur nos 172 photos de test (format JPEG), il trouve une voiture dans 82 % des cas. Il ne trouve rien sur les photos d'intérieur ou les schémas, ce qui est normal : dans ce cas, il n'y a simplement pas de cadre.

Popup après un scan. Quand on scanne une voiture, le cadre rouge se verrouille d'abord sur la photo, puis un popup animé s'ouvre, façon « ouverture de paquet » :

- l'image avec son cadre rouge ;
- la marque en petit, puis le modèle en gros ;
- la période de production ;
- une jauge de confiance qui se remplit (vert si $\geq 70\%$, orange si $\geq 40\%$, rouge en dessous).

Ce popup s'affiche pour tout le monde. En bas : « Ajouter à la bibliothèque » si on est connecté, « Se connecter pour sauvegarder » sinon, et « Fermer » dans les deux cas.

Fiche dans la bibliothèque. Un clic sur une photo ouvre une fiche de même style (« Pokédex ») :

- la photo avec son cadre ;
- la marque et le modèle ;
- la production ;
- la date de découverte ;
- la confiance ;
- un bouton pour supprimer la voiture (avec confirmation).

La page « aperçu » de la semaine dernière a été remplacée par cette fiche.

La base ne garde plus un simple texte de légende mais des champs séparés : marque, modèle, années, confiance et position du cadre. Les 2 photos de test déjà enregistrées ont été converties sans rien supprimer.

Nettoyage du code. On a fusionné la branche IA avec celle de l'application dans une nouvelle branche ia-integration, puis supprimé le code en double et les fichiers inutiles :

Côté Flutter, les parties copiées-collées (en-têtes des pages, mise en page, fenêtre de confirmation, chargement des images, champs des formulaires) ont été regroupées dans des widgets partagés. dart analyze ne signale aucun problème. Le README a été entièrement réécrit (architecture à 4 parties, lancement de l'IA, vérification).

Difficultés rencontrées et solutions :

- Conflits à la fusion : deux fichiers (README.md et .gitignore) existaient dans les deux branches. On a gardé les deux listes dans le .gitignore et déplacé le README de l'IA dans ai/README.md.
- Fichier cassé : dataset_preparer.py ne pouvait pas être utilisé car le init était mal indenté (la classe n'acceptait aucun paramètre). On l'a corrigé.
- Installation de TensorFlow : pip ne trouvait pas la bonne version car le terminal utilisait Python 3.14, alors que TensorFlow 2.21 existe pour Python 3.13. On utilise donc un environnement virtuel en Python 3.13 et on appelle directement son Python, sans l'activer (le script d'activation n'est pas le même dans cmd et dans PowerShell).
- Plusieurs scans en même temps : le serveur peut recevoir plusieurs photos au même moment, mais le détecteur ne sait en traiter qu'une à la fois. Par précaution, on a ajouté un verrou : si deux personnes scannent en même temps, la deuxième photo attend que la première soit terminée.
- Confiance qui semble basse : une voiture correctement reconnue peut avoir seulement 40 à 56 % de confiance. La confiance n'est pas « la probabilité d'avoir raison » mais la part de préférence du modèle pour cette voiture par rapport aux 5 autres. Quand deux voitures se ressemblent (Ferrari F12 et Ford Mustang), cette part se partage.

Limites constatées et suite :

- Le modèle ne connaît que 6 voitures et en répond toujours une, même pour une autre voiture : une Mazda MX-5 est prise pour une Fiat 500 Abarth à 79 %. Il faut un seuil de confiance (« voiture non reconnue »).

Organisation du repo GitHub

- main : README et rapports.
- backend-mongodb : application Flutter et backend (bibliothèque en ligne).
- flutter_dev : ancienne version de l'application.
- test_mobilenet_python_thomas : IA (classes, évaluation, dataset).
- ia-integration (nouvelle) : fusion de l'application et de l'IA, avec tout ce qui est décrit ci-dessus.

Traçabilité de l'usage de l'IA

Cette semaine, l'IA a surtout été utilisée en mode hybride. Nous avons développé et testé nous-mêmes les fonctionnalités, mais nous avons utilisé principalement ChatGPT, Claude et GitHub Copilot pour chercher des solutions à des problèmes que nous ne connaissions pas encore, comprendre leur fonctionnement, et gagner du temps sur certaines tâches répétitives que nous savions déjà réaliser.

Détection de la voiture et cadre rouge

Notre modèle MobileNetV3 permettait de reconnaître le modèle d'une voiture, mais il ne permettait pas de savoir où se trouvait la voiture dans l'image. Nous devions donc trouver une autre solution pour pouvoir dessiner le cadre rouge demandé dans le sujet.

Nous avons donc demandé :

« J'ai une application Flutter qui envoie une photo de voiture à une IA Python. Mon modèle sait reconnaître le modèle de la voiture mais ne sait pas où elle est dans l'image. Comment je peux trouver la position de la voiture pour dessiner un cadre rouge autour ? »

L'IA nous a expliqué la différence entre classification d'image et détection d'objets, et nous a proposé plusieurs solutions, dont l'utilisation d'un modèle SSD MobileNet avec TensorFlow Lite. Après avoir recherché cette solution, nous avons décidé de la tester car elle était légère et déjà capable de détecter la classe « car ».

Nous avons ensuite demandé un exemple plus proche de notre projet :

« Peux-tu me montrer un exemple Python simple pour charger un modèle SSD MobileNet TensorFlow Lite, lui donner une image et récupérer les coordonnées x, y, largeur et hauteur de la voiture détectée ? Explique chaque étape. »

L'exemple nous a surtout servi à comprendre l'utilisation de l'interpréteur TensorFlow Lite, les entrées/sorties du modèle et les coordonnées retournées. Le code proposé n'a pas été repris tel quel, nous l'avons adapté dans notre classe `detector.py`, ajouté la gestion du cas où aucune voiture n'est trouvée et testé le numéro correspondant réellement à la classe voiture, car la première valeur supposée n'était pas correcte avec notre modèle.

Intégration de l'IA avec Flutter et le backend

Pour relier le modèle Python à notre backend Node.js, nous avons utilisé l'IA pour nous aider à choisir une architecture simple. Nous voulions surtout éviter que Flutter communique directement avec plusieurs serveurs.

« Mon application Flutter communique déjà avec un backend Node.js Express. J'ai maintenant un modèle TensorFlow en Python. Quelle architecture simple je peux utiliser pour que Flutter continue à communiquer uniquement avec Node.js ? »

L'IA nous a proposé de mettre le modèle derrière une petite API Python, par exemple avec Flask, puis de laisser le backend Node.js appeler cette API. Nous avons retenu cette solution et créé le service ai/server.py.

Nous avons aussi demandé des exemples simples pour comprendre comment envoyer une image entre les deux serveurs. Les propositions ont ensuite été adaptées à nos routes existantes, notamment pour gérer les erreurs lorsque le serveur IA est éteint. Certaines propositions de l'IA étaient trop complexes pour notre besoin, avec par exemple une file de messages ou plusieurs services supplémentaires, nous ne les avons pas retenues.

Jauge de confiance et popup

Pour l'interface Flutter, une première version de la jauge de confiance avait été faite manuellement. Nous avons ensuite utilisé GitHub Copilot pour améliorer l'animation sans devoir rechercher toute la syntaxe Flutter.

« J'ai déjà cette jauge Flutter qui affiche le pourcentage de confiance de mon modèle. Je voudrais l'améliorer pour que la barre parte de 0 et se remplisse progressivement jusqu'au pourcentage obtenu quand le popup s'ouvre. Je veux garder mon design actuel. Je voudrais que la jauge soit verte à partir de 70 %, orange entre 40 et 70 %, et rouge en dessous de 40 %. Comment faire simplement sans refaire tout mon widget ? »

L'IA nous a proposé notamment d'utiliser les widgets d'animation de Flutter. Nous avons adapté la proposition à notre popup existant et changé plusieurs éléments visuels proposés automatiquement afin de conserver notre style « Pokédex ». La logique des seuils et le design final ont donc été décidés et ajustés par nous.

Nettoyage après la fusion des branches

Après la fusion entre la branche contenant l'application et celle contenant l'IA, le projet contenait plusieurs fichiers en double et du code devenu inutile. Cette tâche pouvait être réalisée manuellement, mais nous avons utilisé l'IA pour accélérer l'analyse.

Nous lui avons donné la structure du projet et demandé par exemple :

« On vient de fusionner deux branches Git d'un même projet. Analyse cette arborescence et indique-moi les fichiers ou dossiers ou des bouts de code qui semblent être des doublons ou qui ne semblent plus utilisés. Ne supprime rien, donne-moi seulement une liste avec la raison. »

Puis, pour Flutter :

« Regarde ces fichiers Flutter et trouve les parties de code répétées plusieurs fois qui pourraient devenir des widgets communs. Ne modifie pas le comportement de l'application. »

Cela nous a permis d'identifier plus rapidement plusieurs éléments, mais nous avons vérifié chaque proposition avant suppression. Par exemple, le dossier contenant une copie complète du projet Flutter, les anciens scripts Python et certains fichiers qui n'avaient plus

d'utilisation ont été supprimés après vérification. À l'inverse, certains fichiers signalés comme potentiellement inutiles par l'IA ont été conservés lorsque nous avons vérifié qu'ils étaient encore utilisés.

Bilan de notre utilisation de l'IA

L'IA n'a donc pas été utilisée pour générer entièrement le projet. Son utilisation cette semaine a surtout eu trois objectifs :

- comprendre une technologie inconnue, par exemple la détection d'objets avec TensorFlow Lite
- obtenir une base ou un exemple, puis l'adapter à notre architecture et la tester ;
- gagner du temps sur des tâches que nous savions faire, comme rechercher du code dupliqué, compléter des widgets Flutter ou nettoyer le projet après la fusion.

Nous avons donc cherché à utiliser l'IA comme outil d'assistance et d'apprentissage, en gardant la compréhension et la validation du code que nous intégrons au projet.

